Personal Reflection Notes: Knowledge Distillation & Cloud Engineering

Student Name: Sabavat Sachith Naik | **Roll Number:** 230101093 | **Course:** CS3081 Cloud Computing
GitHub Repository: https://github.com/Nani779-hub/230101093-cloud-miniproject

1. Conceptual Understanding & Core Learnings

Entering this mini-project, my exposure to Large Language Models was largely confined to high-level prompting via hosted proprietary APIs. This project forced me to peel back the abstraction layers and confront the harsh operational realities of cloud computing, resource constraints, and distributed systems. The overarching task—distilling semantic parsing capabilities from a 3.8-billion parameter teacher model into an ultra-compact 70-million parameter student model, containerizing the resulting artifacts, and orchestrating them on a local Kubernetes cluster—taught me several foundational principles that textbooks rarely convey.

1.1 The Mechanics of Knowledge Distillation (KD)

I learned that Knowledge Distillation is fundamentally an information-compression technique governed by a teacher-student paradigm. Rather than training a compact model from scratch on scarce human-labeled data, KD utilizes a capable teacher model to supervise a smaller, computationally agile student. In our specific setting, we applied *Sequence-Level Knowledge Distillation* (Kim & Rush, 2016). Instead of matching the dense token-level probability distributions (logits) across high-dimensional softmax layers—which would require identical tokenizers, aligned vocabularies, and massive memory bandwidth—the student model is trained strictly on the greedily decoded surface text produced by the teacher: Command: <text>\nJSON: <teacher_reply>.

This sequence-level approach turns a complex distribution-matching problem into a standard conditional causal language modeling task with cross-entropy loss. It allows a student with a completely different vocabulary (Pythia-70M's 50k BPE tokens) to learn from a teacher with an entirely distinct architecture (Phi-3.5's 32k TikToken vocabulary).

1.2 Cloud-Native Architecture & Decoupled Storage

A critical cloud computing tenet reinforced during this project is the strict separation of compute and storage. In traditional naive deployments, developers often bake heavy model weights directly into container images. Through Task 3 and Task 4, I experienced why this is an anti-pattern: it bloats Docker image transfer times, inflates local storage overhead, breaks rapid autoscaling, and tightly couples model versioning with code release cycles. By leveraging SeaweedFS as an S3-compatible object store, our container images remained below 1.65 GB, dynamically pulling model weights from object storage buckets on pod startup using authenticated Kubernetes Secrets.

1.3 Stateless vs. Stateful Network Controls

Designing the subnet allocation table and Network Access Control Lists (Task 5) crystallized the operational difference between stateful firewalls and stateless packet filters. In a stateless NACL, there is no connection state table. Every single packet traveling across the subnet boundary must match an explicit rule. I realized why ephemeral response ports (1024–65535) must be explicitly opened for return traffic, and why strict numerical rule ordering (lowest rule number wins) is crucial to drop malicious CIDRs before permissive web traffic rules are evaluated.

2. Technical Challenges Faced and Engineering Solutions

Working within the tight physical confines of a personal laptop (7.6 GB RAM, CPU-only execution) presented substantial systems-level friction. Overcoming these hurdles demanded disciplined debugging and architectural workarounds.

Challenge 1: CPU Hardware Saturation & Ollama Queue Contention

The Problem: Labelling 3,000 commands using a 3.8B parameter teacher model on pure CPU is computationally demanding. Initially, running parallel worker pods caused severe CPU thread contention and internal Ollama socket timeouts (HTTP 500), with inference times ballooning from 7s to over 45s per record.
The Solution: I investigated the Ollama process internals on Windows and discovered that Ollama's underlying llama-server.exe was initialized with a single concurrency slot (-np 1). Two simultaneous workers sending requests forced one request into an unbounded wait queue. I optimized the worker architecture by implementing exponential backoff retries in worker.py, tuning batch checkpointing to S3 every 10 records, and sequencing background evaluation tasks so they did not compete for memory bus bandwidth.

Challenge 2: PyTorch Dependency Bloat & Container Image Ceiling

The Problem: The assignment imposed a strict 2.0 GB limit on the inference service container image. The default pip install torch downloads over 2.5 GB of pre-compiled NVIDIA CUDA, cuDNN, and NCCL GPU shared libraries, immediately exceeding the image ceiling and triggering out-of-disk failures.
The Solution: In service/Dockerfile, I explicitly pinned CPU-only wheels: pip install torch==2.6.0+cpu --extra-index-url https://download.pytorch.org/whl/cpu. Combined with a multi-stage Python 3.11-slim base and stripping unnecessary development caches, the final image size was reduced to **1.64 GB**, well below the 2.0 GB threshold.

Challenge 3: Zero-Downtime Rolling Updates with Weight Hydration

The Problem: Because container images do not bake in model weights, a newly spun-up pod requires ~14 seconds upon boot to download 280 MB of model weights from SeaweedFS S3 and load them into RAM. During a rolling update from v1 to v2, routing traffic to a new pod before its weights are hydrated would result in widespread HTTP 503 errors and catastrophic SLO violation.
The Solution: I coupled FastAPI's modern lifespan handler with Kubernetes readiness and liveness probes. /healthz reports 200 immediately to signify the Python process is alive, but /readyz strictly returns HTTP 503 until the model weights are fully loaded into memory. In k8s/app/deployment.yaml, setting maxUnavailable: 0 and maxSurge: 1 ensured Kubernetes never terminates an existing healthy replica until the new pod's readiness probe turns HTTP 200, achieving a rolling update error rate of just **0.12%** under full Locust load.

Challenge 4: Windows-WSL-Container Networking Complexity

The Problem: Developing across Windows 11 host, WSL2 Ubuntu, and k3d container networking created split-DNS and IP routing barriers. Pods inside k3d could not resolve localhost:11434 to reach the Windows Ollama instance.
The Solution: I utilized k3d's built-in host bridge routing, directing container workloads to host.k3d.internal:11434, while binding Ollama to 0.0.0.0:11434 with appropriate Windows Firewall exceptions.

3. Empirical Findings, Observations & What Could Be Improved

3.1 Key Experimental Findings & Observations

Finding 1 — Surprising Plasticity of Ultra-Small Models: An untrained Pythia-70M model possesses 0.0% validity on JSON frames; its raw outputs are rambling, incoherent natural language. However, after fine-tuning on just 1,500 teacher-generated examples over 5 epochs, JSON validity skyrocketed to **96.0%**, and full-frame semantic accuracy reached **52.0%**. This proves that even microscopic language models can be successfully adapted to rigid grammar and slot extraction tasks when the output syntax is highly constrained.

Finding 2 — The Teacher Acts as a Double-Edged Sword (Error Propagation): During data cleaning (Task 1), clean.py eliminated 140 flawed teacher predictions (invalid JSON, incorrect keys, out-of-domain intents, or hallucinated slot values). However, subtle semantic errors that complied with the schema escaped detection. Because sequence-level distillation treats teacher output as absolute ground truth, the student model learned several erroneous biases. The student's ceiling is strictly bounded by the teacher's precision.

Finding 3 — Memory & Concurrency Dynamics Under Load: Memory profiling revealed that Pythia-70M consumes only ~480 MiB of RAM at steady state, peaking at **618 MiB** during active multi-user inference under a 1 GiB limit. Under Locust load (4 concurrent users), p50 latency was **380 ms**, while p95 reached **1,420 ms**. The autoscaler (HPA) demonstrated that CPU thresholding (70%) is an effective trigger to scale from 1 to 2 replicas, doubling throughput to 8.4 req/s.

3.2 What Could Have Been Done Better (Critique & Future Scope)

Reflecting critically on my design decisions, several areas could be substantially upgraded in an enterprise setting:

• **Local Node Caching via Kubernetes DaemonSets:** Having every new pod download model weights over HTTP from SeaweedFS introduces a ~14.8 second cold-start latency. In production, a DaemonSet or CSI driver should pre-populate model weights into a hostPath cache on local node NVMe storage, reducing cold starts to sub-second durations.

• **Quantization & High-Throughput CPU Serving:** Serving raw FP32 PyTorch weights on CPU is inefficient. Exporting the fine-tuned Pythia model to ONNX Runtime or utilizing INT8/INT4 quantization with llama.cpp or vLLM would reduce RAM consumption to <200 MiB and double token generation speeds.

• **Multi-Teacher Ensemble Distillation:** Instead of relying on a single teacher (Phi-3.5), distilling from an ensemble (e.g., Llama-3-8B + Mistral-7B) with majority voting would filter out hallucinations before they reach the student training set.

• **Stateful Security Policies:** Supplementing stateless AWS-style NACLs with Kubernetes NetworkPolicies (using Cilium eBPF) would provide granular pod-to-pod mutual TLS and Layer-7 policy enforcement.

3.3 Concluding Thoughts

This project bridged the gap between machine learning theory and distributed cloud systems. Successfully delivering knowledge distillation, S3 object storage integration, zero-downtime rolling deployments, automated CI/CD gating, and Locust load testing in a single coherent pipeline provided an invaluable, comprehensive masterclass in real-world cloud engineering.